

Az elkészített szoftverek

Sándor Balázs

2025. április 24.



Tartalomjegyzék

1. Elkészített szoftverek	2
1.1. A melléklet célja	2
1.2. Képkonverzió	3
1.2.1. convert.bat képkonverziós program.	5
1.2.2. convert-runner.py párhuzamos többmappás képkonvertáló. . .	8
1.3. Átnevezés és ismétlődések szűrése	9
1.3.1. A rename2.py MD5 átnevező szkript.	10

1. fejezet

Elkészített szoftverek

1.1. A melléklet célja

A diplomamunka során elkészített szoftverek központi szerepet játszottak az adatfeldolgozásban, a tanítási és tesztelési folyamatok automatizálásában, valamint a mért adatok kiértékelésében. A teljes munkafolyamat Python nyelven készült, Jupyter notebookok, PyTorch keretrendszer, valamint a matplotlib, seaborn, pandas és numpy könyvtárak használatával. A modellek betanítása és kiértékelése saját fejlesztésű kódbázison történt, amely lehetővé tette az egyedi igények szerinti metrika- és grafikon-előállítását.

A fejlesztett szoftvermodulok a következő főbb komponensekre bonthatók:

Adat-előkészítő és konvertáló eszközök: Ezek a programrészek a nyers fotóállományból készítettek előfeldolgozott, egységesített és annotált képadatbázist. Ide tartozik az EXIF-adatok megőrzését biztosító átalakítás, az MD hash alapú elnevezés és ismétlődés szűrés, az automatikus kategorizálás, a megfelelő képméretre vágás, tömörítés, valamint a tanító/tesztelő/validáló halmazok létrehozása.

A modellek tanítását vezérlő szkriptek: A tanítórendszer lehetővé teszi különféle modellek (ResNet50, DenseNet121, EfficientNet-B0, ConvNeXt-Tiny, stb.) betöltését, finomhangolását, valamint többféle tanítási fázis (pl. teljes tanítóhalmaz, iteratív bővítés) automatizálását. A szkriptek rögzítik az összes fontos tanítási metrikát (loss, accuracy, precision, recall, F1-score), epochonként és osztályszinten.

Tesztelési és kiértékelő modulok: Ezek a programok lehetővé teszik a betanított vagy gyári súlyokkal rendelkező modellek teljesítményének összehasonlítását különféle metrikák alapján, mind Top-1, Top-5, mind osztályszpecifikus pontosság szerint.

A program automatikusan készít confusion matrixokat, osztályeloszlás-görbéket, metrika-idő vagy metrika-mintaszám függvényeket.

Vizualizációs alrendszer: A metrikák kiértékelését követően a rendszer képes automatikusan egységes formátumú és jól olvasható grafikonokat generálni. A matplotlib és seaborn könyvtárakkal készült diagramok a tanítási folyamat vizuális dokumentációját nyújtják, megkönnyítve az összehasonlítást.

Egyedi fejlesztésű kis modell és fejrész prototípusok: A szoftvercsomag része az erőforrás-kímélő, ResNet18-alapú lokális modell fejlesztését támogató kódbasis is, amely lehetővé teszi különféle hálózati „fejek” tesztelését és kiértékelését. Ezek külön paramétereizhetők és egységes módon mérhetők a többi modellhez hasonlóan. Minden programrész jól dokumentált, verziókövetett formában készült, a főbb beállítások és paraméterek külön konfigurációs blokkokban találhatók a notebook-okban. A virtualizált környezet kialakításáért felelős telepítő blokkok is elhelyezésre kerültek a notebook-okba a könnyebb megismételhetőség érdekében. A melléklet tartalmazza a szoftverrendszer logikai felépítésének leírását, a legfontosabb fájlok magyarázatát, valamint a kód működésének szemléltetését példákkal.

Az elkészített programkódok a projekt GitHub könyvtárában találhatóak.

A projekt GitHub repository-ja:

<https://github.com/SandorBalazsHU/elte-ik-msc-thesis>

1.2. Képkonverzió

A képkonverziót egy saját fejlesztésű, `convert.bat + convert_runner.py` nevű szkript végezte, amely Windows batch és Python alapon működő, multimappás kötegelt feldolgozásra képes rendszerként lett kialakítva. Feladata a képek egységes átméretezése 224×224 pixelre, valamint megfelelő tömörítéssel történő konvertálása JPG formátumba.

A folyamat első lépéseként a szkript listázza az adott mappában található képfájlokat, majd minden képre külön feldolgozási folyamatot indít, két lépésben:

- Az **IrfanView** eszközzel történik a nyers képfájlok (pl. NEF formátum) konvertálása és méretezése.
- Ezt követően az **ImageMagick** segítségével történik a középpontos, 1:1 oldalarányú képkivágás.

A két eszköz kombinálására azért volt szükség, mert:

- Az `ImageMagick` nem minden esetben kezelte megbízhatóan a `NEF` formátumot,
- Az `IrfanView` viszont nem biztosított precíz középpontos kivágást.

A szkript futtatását egy Python program végezte, amely rekurzívan indította el a `convert.bat` szkriptet minden egyes almappára, lehetővé téve ezzel a több szálon futó, párhuzamos konverziót.

A rendszer teljesítménye rendkívül hatékornak bizonyult: a többmagos CPU-teljesítményt teljes mértékben kihasználta, és magas képfeldolgozási sebességet ért el. Az alábbi táblázatok részletesen bemutatják az egyes műveletek időigényét és a teljes rendszer átlagos teljesítményét.

1.1. táblázat. Képkonverziós műveletek időigénye és optimalizálhatósága

Művelet	Átlagos idő	Lehetőség a gyorsításra?
NEF → JPG konverzió	~35–45 ms	IrfanView optimalizálva
Átméretezés	~10–20 ms	ImageMagick CPU-teljesítményen
MD5 hash / átnevezés	<5 ms	Pythonban villámgyors
Kézi overhead	~2–3 s / futás	Teljesen automatizálható

1.2. táblázat. Átlagos feldolgozási teljesítmény

Sebességmutató	Érték
Képenkénti feldolgozási idő	52,6 ms
Átlagos képfeldolgozási sebesség	~19,0 kép / másodperc
Konverziós sávszélesség (becsült)	~1,27 MB/s (65 KB/kép mellett)
10 000 kép becsült ideje	~10,4 perc
150 000 kép becsült ideje	~2 óra 36 perc

A következő ábra egy mappa feldolgozását mutatja be, amely során a `convert.bat` és a `convert_runner.py` működött együtt a konverziós feladatok elvégzésére:

```
[RUN] D:\WORK\DVD10\Nyaralások 2\Repülőlőnapok\Vári Gyula és a kalocsai repülőlőnap 2007\convert.bat
[INFO] Creating 'resized' folder...

[STEP 1] Counting input files...
Found 14 images to process.

[STEP 2] Converting all to resized JPG (short side = 224 px)...
[Converting] 7% (1 / 14)
[Converting] 14% (2 / 14)
[Converting] 21% (3 / 14)
[Converting] 28% (4 / 14)
[Converting] 35% (5 / 14)
[Converting] 42% (6 / 14)
[Converting] 50% (7 / 14)
[Converting] 57% (8 / 14)
[Converting] 64% (9 / 14)
[Converting] 71% (10 / 14)
[Converting] 78% (11 / 14)
[Converting] 85% (12 / 14)
[Converting] 92% (13 / 14)
[Converting] 100% (14 / 14)

Waiting for all IrfanView conversions to finish...

[STEP 3] Cropping to center 224x224 (parallel)...
[Cropping] 7% (1 / 14)
[Cropping] 14% (2 / 14)
[Cropping] 21% (3 / 14)
[Cropping] 28% (4 / 14)
[Cropping] 35% (5 / 14)
[Cropping] 42% (6 / 14)
[Cropping] 50% (7 / 14)
[Cropping] 57% (8 / 14)
[Cropping] 64% (9 / 14)
[Cropping] 71% (10 / 14)
[Cropping] 78% (11 / 14)
[Cropping] 85% (12 / 14)
[Cropping] 92% (13 / 14)
[Cropping] 100% (14 / 14)

Waiting for all cropping to finish...

[DONE] All images processed to 224x224 JPG in 'resized' folder.
```

1.1. ábra. A convert.bat és convert_runner.py futása egy mappa feldolgozására.

1.2.1. convert.bat képkonverziós program.

```
1      @echo off
2      setlocal enabledelayedexpansion
3
4      :: ----- Settings -----
5      set "iview=C:\Program Files\IrfanView\i_view64.exe"
6      set "magick=magick"
7      set "inputFolder=%~dp0"
8      set "outputFolder=%inputFolder%resized"
9
10     :: ----- Create output folder -----
11
12     if not exist "%outputFolder%" (
13         echo [INFO] Creating 'resized' folder...
14         mkdir "%outputFolder%"
```

```
14      )
15
16      :: ----- Count input files
17      -----
18      echo.
19      echo [STEP 1] Counting input files...
20
21      set /a count=0
22      for %%f in ("%inputFolder%*.nef") do set /a count+=1
23      for %%f in ("%inputFolder%*.jpg") do set /a count+=1
24
25      echo Found !count! images to process.
26
27      :: ----- Resize in parallel
28      -----
29      echo.
30      echo [STEP 2] Converting all to resized JPG (short side = 224
31      px)...
32
33      set /a i=0
34      for %%f in ("%inputFolder%*.nef") do (
35          set /a i+=1
36          start /b "" "%iview%" "%f" /resize_short=224 /
37              aspectratio /resample /jpgq=80 /convert="%outputFolder
38              %\%%~nf_nef.jpg"
39          call :progress !i! !count! "Converting"
40      )
41
42      for %%f in ("%inputFolder%*.jpg") do (
43          set /a i+=1
44          start /b "" "%iview%" "%f" /resize_short=224 /
45              aspectratio /resample /jpgq=80 /convert="%outputFolder
46              %\%%~nf_jpg.jpg"
47          call :progress !i! !count! "Converting"
48      )
49
50      echo.
51      echo Waiting for all IrfanView conversions to finish...
52      call :wait_irfan
53
54      :: ----- Crop in parallel -----
```

```

48     echo.
49     echo [STEP 3] Cropping to center 224x224 (parallel)...
50
51     set /a totalCrop=0
52     for %%f in ("%outputFolder%\*.jpg") do (
53         set /a totalCrop+=1
54     )
55
56     set /a i=0
57     for %%f in ("%outputFolder%\*.jpg") do (
58         set /a i+=1
59         start /b "" cmd /c %magick% "%%f" -gravity center -crop
60             224x224+0+0 +repage "%%f"
61         call :progress !i! !totalCrop! "Cropping"
62     )
63
64     echo.
65     echo Waiting for all cropping to finish...
66     call :wait_magick
67
68     echo.
69     echo [DONE] All images processed to 224x224 JPG in 'resized'
70     folder.
71     endllocal
72     pause
73     exit /b
74
75     :: ----- Progress display -----
76     :progress
77     setlocal
78     set "current=%~1"
79     set "total=%~2"
80     set "label=%~3"
81     set /a percent=(current*100)/total
82     echo [%label%] !percent!%% (!current! / !total!)
83     endllocal
84     exit /b
85
86     :: ----- Wait for IrfanView jobs to finish -----
87     :wait_irfan

```



```
86     tasklist | find /I "i_view64.exe" >nul
87     if not errorlevel 1 (
88         timeout /t 1 >nul
89         goto wait_irfan
90     )
91     exit /b
92
93     :: ----- Wait for ImageMagick jobs to finish
94     -----
95     :wait_magick
96     tasklist | find /I "magick.exe" >nul
97     if not errorlevel 1 (
98         timeout /t 1 >nul
99         goto wait_magick
100    )
101    exit /b
```

1.1. forráskód. convert.bat párhuzamos egy mappán belüli képkonverziós szkript.

1.2.2. convert-runner.py párhuzamos többmappás képkonver-táló.

```
1     import os
2     import shutil
3     import subprocess
4     import sys
5
6     def main():
7         # A szkript könyvtára
8         base_dir = os.path.dirname(os.path.abspath(__file__))
9         convert_bat = os.path.join(base_dir, 'convert.bat')
10
11         if not os.path.isfile(convert_bat):
12             print(f"[ERROR] 'convert.bat' nincs megtalálható a {
13                 base_dir} könyvtárban.")
14             sys.exit(1)
15
16         # 1) Mappaterkep keszítése (előzetesen, hogy a későbbi '
17             resized' mappákat ne találja meg)
18         directories = []
```

```
17     for root, dirs, _ in os.walk(base_dir):
18         # Ha a mappa neve 'resized', ugorjuk at
19         if os.path.basename(root).lower() == 'resized':
20             continue
21         directories.append(root)
22
23     print(f"[INFO] Felderített {len(directories)} könyvtarat.
24         ")
25
26     # 2) convert.bat masolasa minden mappaba
27     for d in directories:
28         dest = os.path.join(d, 'convert.bat')
29         try:
30             shutil.copy2(convert_bat, dest)
31             print(f"[COPY] {dest}")
32         except Exception as e:
33             print(f"[ERROR] Nem sikerult masolni ide: {d} ({e}
34                 })")
35
36     # 3) A batch fajl futtatasa minden mappaban
37     for d in directories:
38         bat_path = os.path.join(d, 'convert.bat')
39         print(f"[RUN] {bat_path}")
40         # Windows-on futtatashoz
41         subprocess.run(['cmd', '/c', bat_path], cwd=d)
42
43     if __name__ == '__main__':
44         main()
```

1.2. forráskód. convert-runner.py párhuzamos sokmappás képkonverziós szkript.

1.3. Átnevezés és ismétlődések szűrése

A fájlok MD5 hash alapján történő átnevezése kulcsfontosságú lépés volt az adatbázis egységességének és redundancia-mentességének biztosítása érdekében. Mivel minden egyes már konvertált és átméretezett kép a saját MD5 hash értékét kapta fájlnevként, gyakorlatilag kizárhatóvá vált, hogy két azonos kép duplikáltan szerepeljen az adatbázisban.

Ez a művelet azonban a nagy fájlszám és a sok mappa miatt nem volt triviális, és egy hatékony, párhuzamos feldolgozásra képes szkript megírását igényelte. E célra két Python szkriptet fejlesztettem: `rename.py` és `rename2.py`. Ezek a programok rekurzívan végigjárták a teljes mappaszerkezetet, és 50 képenként új szálát indítottak, így biztosítva a feldolgozás párhuzamosságát. Minden szál az adott képcsoportot hash-elte, majd az eredmény alapján nevezte át őket.

A megközelítés jelentős sebességnövekedést eredményezett, bár még így is kissé lassabbnak bizonyult, mint a konverziós-átméretező szkript, amely a rendszer egyik leggyorsabb komponensének bizonyult.

Egy átnevezési példa:

DSC0123.jpg → 9f6e3148b22b4e13a7b52b66f99fd912.jpg

Az átnevezési szkript futása

A következő ábra egy konkrét futtatást mutat be, ahol a `rename.py` szkript egy mappa képeit hash-eli és nevezi át párhuzamos szálakon keresztül:



1.2. ábra. A `rename.py` szkript futása.

1.3.1. A `rename2.py` MD5 átnevező szkript.

```

1 import os
2 import random
3 import string
4 import hashlib
5 from tqdm import tqdm

```

```

6
7 # --- Beallitasok ---
8 NAME_LENGTH = 25 # Valtoztathato
9     nevhossz (pl. 20-25)
10 USE_MD5 = True # Ha True, az uj nev
11     az MD5 hash lesz
12 TARGET_EXTENSIONS = {'.jpg', '.jpeg', '.png'} #
13     Feldolgozando kiterjesztések
14
15 # A szkript könyvtara lesz az alapertelmezett gyoker
16 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
17
18 # --- Random nev generalasa ---
19 def generate_random_name(length):
20     chars = string.ascii_letters + string.digits
21     return ''.join(random.choices(chars, k=length))
22
23 # --- MD5 hash generalasa ---
24 def generate_md5(filepath):
25     with open(filepath, 'rb') as f:
26         return hashlib.md5(f.read()).hexdigest()
27
28 # --- Könyvtarak felderítése ---
29 directories = []
30 for root, dirs, _ in os.walk(BASE_DIR):
31     directories.append(root)
32
33 print(f"[INFO] Felderített könyvtarak száma: {len(directories)}")
34
35 for d in directories:
36     print(f" - {os.path.relpath(d, BASE_DIR)}")
37
38 # --- Minden könyvtár feldolgozása ---
39 for FOLDER in directories:
40     # Fájlok listázása a célkiterjesztésekkel
41     files = [
42         f for f in os.listdir(FOLDER)
43         if os.path.isfile(os.path.join(FOLDER, f))
44         and os.path.splitext(f)[1].lower() in
45             TARGET_EXTENSIONS
46     ]

```

```

42
43     if not files:
44         print(f"[INFO] {os.path.relpath(FOLDER, BASE_DIR)}:
45             nincs megfelelo fajl.")
46         continue
47
48     print(f"[INFO] {os.path.relpath(FOLDER, BASE_DIR)}: {len(
49         files)} fajl atnevezese...")
50
51     for filename in tqdm(files, desc=f"Renaming in {os.path.
52         relpath(FOLDER, BASE_DIR)}"):
53         full_path = os.path.join(FOLDER, filename)
54         ext = os.path.splitext(filename)[1].lower()
55
56         # uj nev generalasa
57         if USE_MD5:
58             newname = generate_md5(full_path)
59         else:
60             newname = generate_random_name(NAME_LENGTH)
61
62         new_filename = f"{newname}{ext}"
63         new_path = os.path.join(FOLDER, new_filename)
64
65         # utkozes elkerulese
66         while os.path.exists(new_path):
67             if USE_MD5:
68                 newname += '_1'
69             else:
70                 newname = generate_random_name(NAME_LENGTH)
71                 new_filename = f"{newname}{ext}"
72                 new_path = os.path.join(FOLDER, new_filename)
73
74         os.rename(full_path, new_path)
75
76     print(f"[DONE] {os.path.relpath(FOLDER, BASE_DIR)}:
77         minden fajl atnevezve.")
78
79     print("[ALL DONE] Az osszes könyvtarban vegezve.")

```

1.3. forráskód. A rename2.py MD5 átnevező szkript.

Ábrák jegyzéke

1.1. A <code>convert.bat</code> és <code>convert_runner.py</code> futása egy mappa feldolgozása.	5
1.2. A <code>rename.py</code> szkript futása.	10

Táblázatok jegyzéke

1.1. Képkonverziós műveletek időigénye és optimalizálhatósága	4
1.2. Átlagos feldolgozási teljesítmény	4